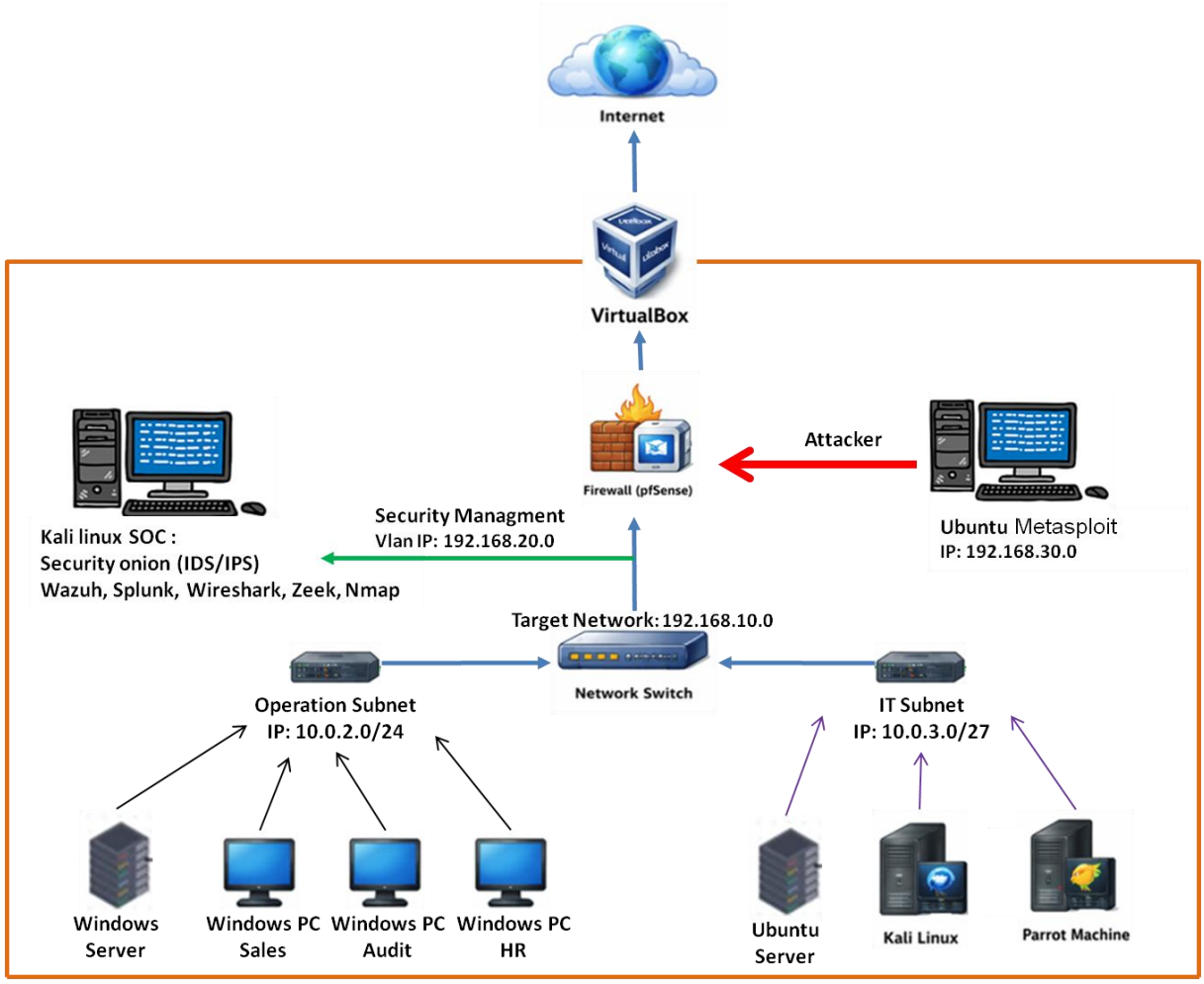
Advanced SOC Lab Architecture (Enterprise-Style Network)



---

1. Proper network segmentation

Separated:

- **Target network (192.168.10.0)**
- **SOC / Security VLAN (192.168.20.0)**
- **Attacker network (192.168.30.0)**
- **Operational + IT subnets (10.0.x.x ranges)**

It mirrors real enterprise zoning (user LAN, management, security monitoring, red team).

---

2. Dedicated SOC stack

Including:

- Security Onion (IDS/IPS)
- Wazuh (SIEM/XDR)
- Splunk

- Zeek, Wireshark, Nmap

That’s a detection stack. covering:

- Network visibility (Zeek, Security Onion)
- Endpoint/log analysis (Wazuh)
- SIEM correlation (Splunk)

Exactly how blue teams operate.

---

3. Firewall placement (pfSense)

Placing pfSense between:

- attacker
- internal networks
- internet

Choke point and inspection layer.

---

4. Red team simulation

Having a dedicated:

- Ubuntu + Metasploit attacker node

Good move. This allows:

- controlled attack simulation
- testing detections realistically

---

5. Mixed OS environment

Included:

- Windows Server + endpoints
- Linux servers
- Kali + Parrot

This diversity is critical for:

- realistic attack surfaces
- varied telemetry

Gaps / Risks / Things to Improve

1. IP scheme inconsistency

mixing:

- `192.168.x.x` (RFC1918 class C)
- `10.x.x.x` (RFC1918 class A)

- It's unclear how routing is handled between 192.168.10.0 and 10.0.x.x
- Your "Target Network" vs "Operation/IT subnets" relationship is ambiguous

Suggestion and Improvement:

- Either:
  - Make 10.0.x.x subnets **inside** the target network
  - Or define routing explicitly via pfSense VLAN interfaces

---

2. VLAN design not fully enforced

VLANs (good), but:

- No explicit trunking shown
- No VLAN IDs
- No clear L2/L3 boundary definition

Improve by defining:

- VLAN 10 → Users (10.0.2.0/24)
- VLAN 20 → IT (10.0.3.0/27)
- VLAN 30 → SOC (192.168.20.0/24)
- VLAN 40 → Attacker (192.168.30.0/24)

Ensure:

- pfSense handles inter-VLAN routing
- switch is VLAN-aware (even if virtual)

---

3. Detection visibility gaps

Right now:

- SOC is connected, but not clearly positioned for **traffic mirroring**

Key question:
How does Security Onion see traffic?

If no SPAN/mirror:

- IDS visibility will be limited

 Fix:

- Add a **SPAN port / virtual tap**
- Mirror:
    - traffic from switch → Security Onion

---

4. No identity / directory services

I have Windows Server, but not explicitly:

- Active Directory
- DNS / DHCP roles

This is a huge opportunity:

- Add AD DS
- Use it for:
    - authentication attacks (Kerberoasting, Pass-the-Hash)
    - log generation (event logs → Wazuh/Splunk)

---

5. Firewall rules not defined

Architecture is good, but security depends on:

- rules, not topology

Should define:

- Attacker → Target: partially allowed (for testing)
- Target → SOC: allowed (logs)
- SOC → Target: restricted
- IT ↔ Operation: controlled

---

6. No EDR simulation

SIEM but no strong endpoint detection simulation.

Add:

- Wazuh agents on endpoints
- Sysmon on Windows machines

This massively improves:

- detection realism
- telemetry depth

---

7. Internet access control unclear

The advanced Home Lab shows:
Internet → VirtualBox → Firewall

But:

- Are internal machines NATed?
- Is outbound filtering enabled?

Suggest:

- Enable outbound filtering rules
- Simulate:
    - malware beaconing
    - DNS exfiltration

High-impact upgrades (priority list)

To level this lab up fast:

1. Add Active Directory

- Domain Controller
- Join all Windows machines
- Central authentication + logs

2. Implement traffic mirroring

- Feed Security Onion properly
- Otherwise IDS is blind

3. Add Sysmon + Wazuh agents

- Endpoint telemetry = huge improvement

4. Define pfSense rules explicitly

- Turn this into a policy-driven network

5. Simulate real attacks

- Lateral movement
- Privilege escalation
- C2 traffic

Overall assessment

Design maturity: 8/10

Beyond “home lab” level and approaching:

- SOC simulation environment
- Blue vs Red lab

What’s missing is not structure—it’s:

- telemetry flow
- identity layer
- policy enforcement